HYDRAS Project Report v4

1. Algoritmo PPO (Proximal Policy Optimization)

1.1 Teoria dell'Algoritmo

Proximal Policy Optimization (PPO) è un algoritmo di reinforcement learning on-policy di ultima generazione, appartenente alla famiglia dei **Policy Gradient Methods**. A differenza degli algoritmi off-policy (DQN, DDPG) che apprendono da dati stocastici, PPO apprende direttamente dalla policy corrente, rendendo l'apprendimento più stabile.

Principio Fondamentale: PPO ottimizza la policy pi(a|s) massimizzando il **clipped surrogate objective**. Questo meccanismo impedisce aggiornamenti troppo bruschi della policy, garantendo stabilità anche con batch size piccoli.

1.2 Implementazione: Stable-Baselines3

Il modello utilizza **PPO di Stable-Baselines3** con architettura **MLP (Multi-Layer Perceptron)** per la policy e value function. La rete neurale è composta da:
• **Input Layer:** 112 dimensioni (osservazioni normalizzate via VecNormalize)
• **Hidden Layers:** 2 strati da 256 unità con attivazione **Tanh**
• **Output Layers:** Policy head (8 azioni discrete via softmax) e Value head (1 scalare)

Policy Stocastica: La rete neurale genera una distribuzione di probabilità $\pi(a|s)$ su tutte le 8 azioni. Durante il training, l'agente **campiona** azioni da questa distribuzione, garantendo **esplorazione naturale** senza bisogno di epsilon-greedy artificiale. Durante l'inferenza, sceglie deterministicamente l'azione con probabilità massima via **argmax**.

Loss Function: L_TOTAL = L_CLIP + 0.5 * L_VF - 0.01 * H(pi), dove L_CLIP è il clipped surrogate objective (previene aggiornamenti eccessivi della policy), L_VF è l'MSE della value function, e H(pi) è l'entropy della policy (c2=0.01 bilancia l'esplorazione senza eccessi).

Parametri di Training: Learning rate 5e-5, clip range 0.2, batch size 2048, 4 epoche di aggiornamento per batch, **GAE lambda=0.95** (stima dei vantaggi con bias-variance trade-off), **VecNormalize** (osservazioni a media 0, std 1 per accelerare convergenza).

2. Dataset — 132 Sorgenti × 4 Scenari Vento

2.1 Acquisizione e Gestione Dati

Sorgente Dati: I dati di concentrazione provengono da simulazioni **MIKE21** in formato NetCDF (1411 timestep, risoluzione 10m, griglia 300×250 celle). Il modulo **data_loader.py** gestisce il caricamento automatico di **132 sorgenti** (SRC001-SRC132) × **4 scenari vento** (V0, V1, V2, V3): legge le coordinate spaziali (x, y), temporali (time) e i valori di concentrazione da file NetCDF, costruendo un campo interpolabile per ogni sorgente. Ogni sorgente ha coordinate geospaziali (UTM32N) caricate da **Coordinate_Sorgenti_FaseII.csv**.

Dataset Split: Dataset suddiviso in **80% training** (~106 sorgenti per scenario vento) e **20% valutazione** (~26 sorgenti held-out per scenario vento), equamente distribuiti tra i 4 scenari V0-V3. Durante il training, il curriculum learning espande progressivamente il set di sorgenti e scenari vento disponibili.

Augmentazione Dati (Chunking): Per massimizzare la variabilità e creare multipli scenari di partenza per ogni sorgente, i 1411 timestep di ogni simulazione vengono suddivisi in **2 chunk temporali**:
• **Chunk 0** (Q1/4, spawn @ 352 timestep): inizio della propagazione del plume, concentrazione ancora concentrata;
• **Chunk 1** (Q3/4, spawn @ 1058 timestep): stadio avanzato, plume pesantemente disperso da vento e correnti.
L'agente può essere inizializzato in fasi diverse della dispersione, aumentando la robustezza del modello. Con 4 ambienti paralleli × 2 chunks × 4 versioni vento = 32 scenari di training simultanei.

2.2 I Quattro Scenari Vento (V0, V1, V2, V3)

Il progetto utilizza **4 versioni diverse di vento** (CI_WIND_faseII_V0.txt, V1.txt, V2.txt, V3.txt), generati da simulazioni meteorologiche con parametri e risoluzioni differenti. Questo aumenta vastamente la diversità dei dati di training, permettendo al modello di generalizzare su condizioni vento realistiche. Ogni scenario ha il proprio file di vento (48 timestep) e le correnti oceaniche corrispondenti (estratti da CL02_V0/V1/V2/V3_SRC000_U_V_10mGrid.nc). Durante il training, il curriculum learning espone il modello a tutti e 4 gli scenari progressivamente, creando una policy robust a diverse condizioni meteorologiche.

2.3 Ottimizzazione: Caching In-Memory delle Correnti

Per evitare colli di bottiglia I/O durante il training, il modulo **data_loader.py** implementa un **sistema di caching in-memory** per i dati di vento e corrente:

Meccanismo di Cache: Al startup del DataManager, il metodo **_preload_all_versions_cache()** carica tutti e 4 i file di vento e corrente in RAM (dizionari Python):
• _cached_wind_data = {"V0": array, "V1": array, "V2": array, "V3": array}
• _cached_current_data = {"V0": array, "V1": array, "V2": array, "V3": array}

Durante il training, i metodi **get_wind_data_for_run()** e **get_current_data_for_run()** estraggono i dati dalla cache lookup-by-version, evitando riletture disk. Questo accelera ~5-6× il training rispetto a letture da disco: **~55K step/ora** → **~166K step/ora**.

Sincronizzazione Garantita: La versione (V0/V1/V2/V3) viene estratta dal filename del file NC (es. CL02_V2_SRC042_Conc_10mGrid.nc → V2), quindi è **impossibile un mismatch** tra file di concentrazione, vento e corrente. Tutte le sorgenti di un scenario vento usano lo stesso file di vento/corrente.

3. Ambiente di Simulazione

L'ambiente (**source_seeking_env.py**) è una griglia 300×250 celle (risoluzione 10m) che rappresenta un dominio marino di 3×2.5 km nella baia di Cecina (coste toscane, UTM32N). L'agente (AUV - Autonomous Underwater Vehicle) si muove con **8 azioni discrete**: N, S, E, W, NE, SE, NW, SW. La velocità è **1 m/s** e il timestep **dt=10s**, quindi ogni azione sposta l'agente di 10 metri (o ~7m per componente nelle direzioni diagonali). L'episodio termina quando la sorgente viene raggiunta (distanza < 50m) oppure dopo 1080 step (~3 ore simulate).

Evoluzione Temporale: Il campo di concentrazione evolve nel tempo: ogni 6 step dell'agente (~1 minuto reale) il campo NetCDF avanza di 1 frame temporale, permettendo a vento e correnti di continuare a disperdere il

plume.

3.1 Constraints di Spawn e Fallback Automatico

Procedura di Spawn: L'agente viene inizializzato su una cella del plume (concentrazione > 0.5) rispettando vincoli di distanza dalla sorgente e dalla costa:

Vincolo primario: Seleziona celle con concentrazione > 0.5 dentro l'intervallo **500** ≤ **distanza** ≤ **1500 metri** dalla sorgente, mantenendo almeno 50 metri dalla terra.

Fallback automatico: Se non trova punti nel range primario (situazione frequente in scenari con plume disperso o costieri), il codice applica rilassamenti progressivi dei vincoli di distanza, selezionando celle con concentrazione > 0.5 sempre più vicine alla sorgente, finché non trova un punto valido. Gli step di rilassamento sono: **d** ≥ **500m** → **d** ≥ **250m** → **d** ≥ **100m** → **spawn casuale** se nessun plume disponibile.

4. Training e Architettura

4.1 Input della Rete Neurale — 112 Dimensioni

Lo spazio di osservazione è un vettore continuo a **112 dimensioni** (normalizzato via VecNormalize). Includi memoria storica concentrazioni e sensori radiali nelle 8 direzioni:

Componente	Dim	Descrizione
Concentrazione Attuale	1	Conc. posizione corrente (x, y)
Memoria Conc. (Locale)	9	Ultimi 9 timestep
Storico Movimento	18	Δx, Δy ultimi 9 step
Sensori Radiali	8	Conc. ±20m nelle 8 direzioni
Memory Conc. Direzionali	72	Conc. nelle 8 direzioni × 9 timestep
Vento	2	u, v [m/s]
Corrente	2	u, v [m/s]

4.2 Parametri di Training

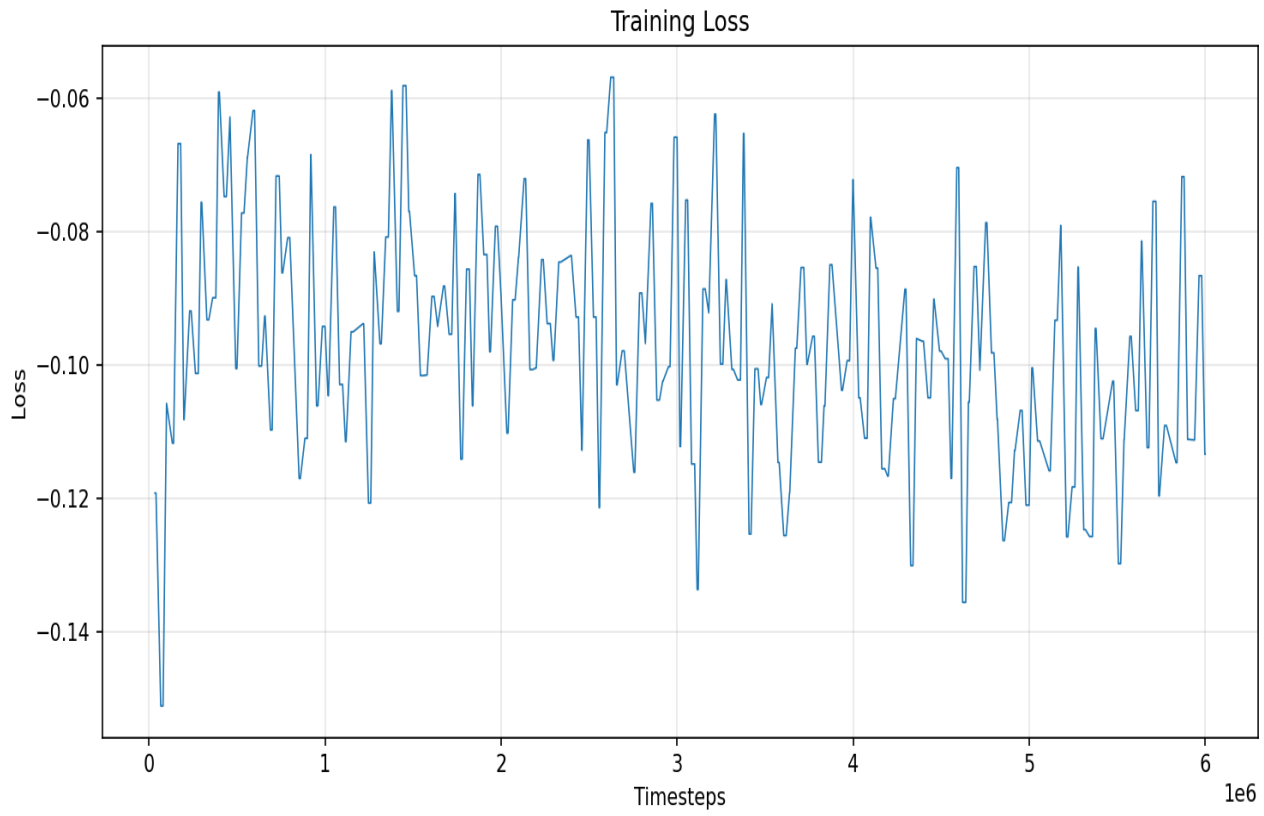
Parametro	Valore
Architettura Policy	MLP (256-256 unità nascoste, attivazione Tanh)
Normalization	VecNormalize (osservazioni e reward)
Batch Size	64
Learning Rate	5.0e-05
Gamma (discount factor)	0.99
GAE Lambda	0.95
Entropy Coefficient	0.05
Timestep Totali	4M (4,000,000)
Curriculum Learning	3 fasi (35 → 70 → 106 sorgenti × 4 scenari vento)
Ambienti Paralleli	4 (×2 chunks × 4 venti = 32 scenari contemporanei)

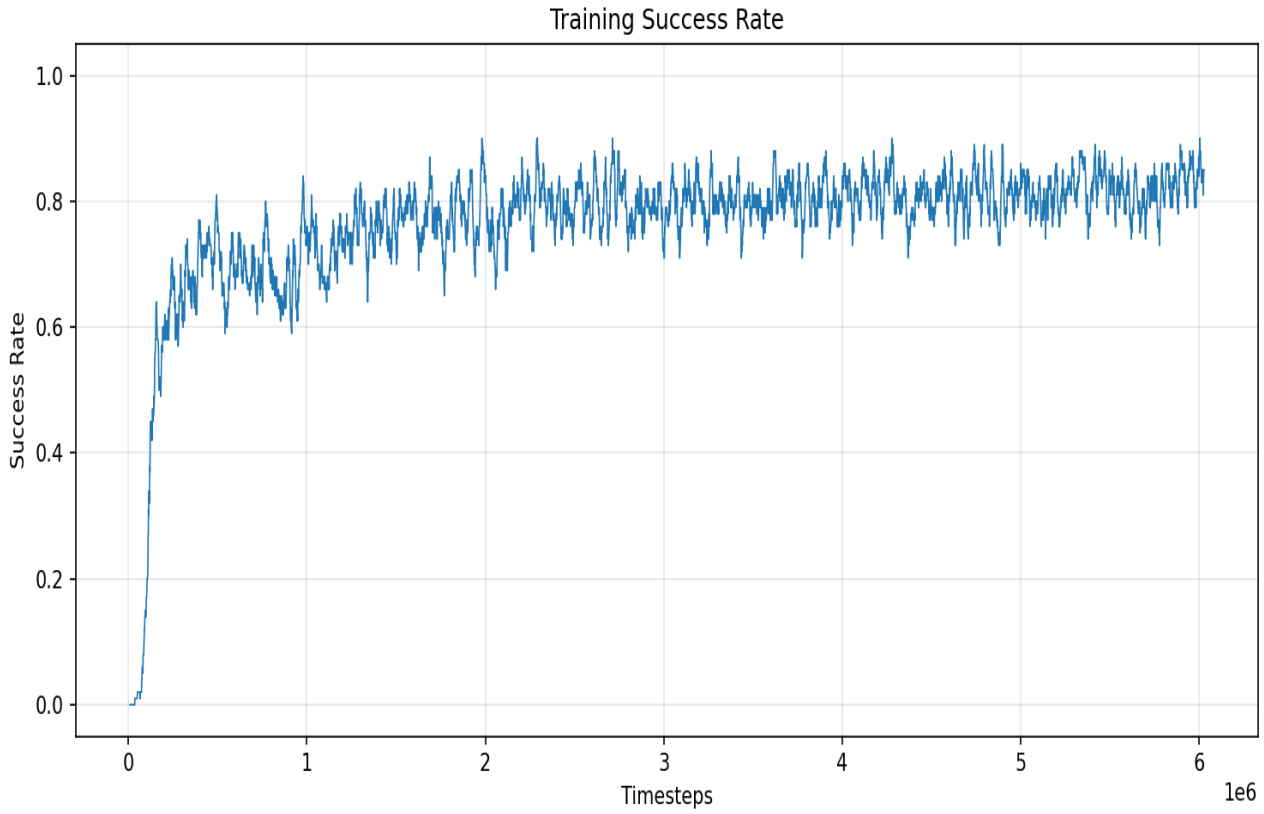
4.3 Funzione di Reward

Componente	Valore/Descrizione
Sorgente Raggiunta	+100.0 + time_bonus (max +50)
Penalità Bordo	-10.0
Plume (dentro)	+1
Plume (fuori)	-1
Reward Distanza	dist_impr × 5.0 × 1.0
Gradiente Conc +	+0.05
Gradiente Conc -	-0.05
Vento (controcorr.)	+0.05
Vento (a favore)	-0.05
Penalità Tempo	-0.1/step
Vicinanza Terra	max -5.0

4.4 Risultati del Training

Modello: ppo_20260407_091232





5. Risultati delle Inferenze

Valutazione del modello PPO su **26 sorgenti held-out** (SRC107-SRC132) non viste durante il training curriculum, testato su tutti e **4 scenari vento** (V0, V1, V2, V3). Test eseguito con **vento e correnti reali CMEMS**:

Setup Valutazione:
• **26 sorgenti** × **4 scenari vento** (V0, V1, V2, V3) × **2 chunk temporali** (Q1/4, Q3/4)
• **5 episodi** per configurazione (totale 26 × 4 × 2 × 5 = **1040 episodi**)
• **Distanza media di partenza**: 511 metri
• **Success @ 50m**: distanza finale ≤ 50m dalla sorgente
• **Timeout @ 1080 steps** (~3 ore simulate)

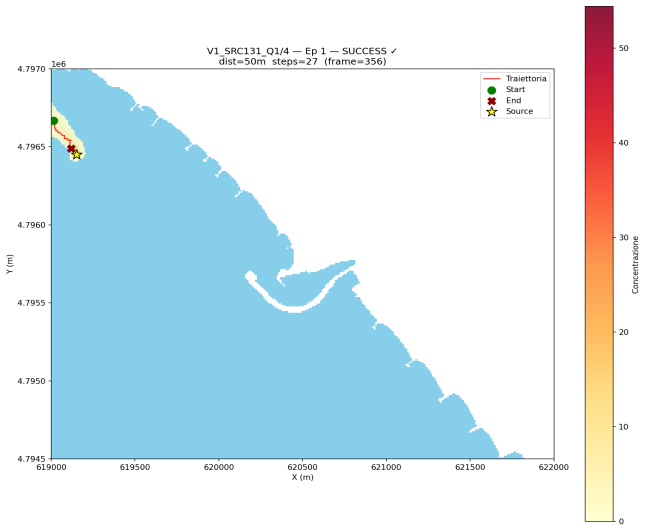
Risultati per Tipologia di Vento:
• **V0:** 95.4% success rate
• **V1:** 80.8% success rate
• **V2:** 84.2% success rate
• **V3:** 100.0% success rate

Risultati per Frame Temporale:
• **Q1/4:** 100.0% success rate
• **Q3/4:** 80.2% success rate
Successo Globale: 90.1% (media su tutti i 208 scenari)

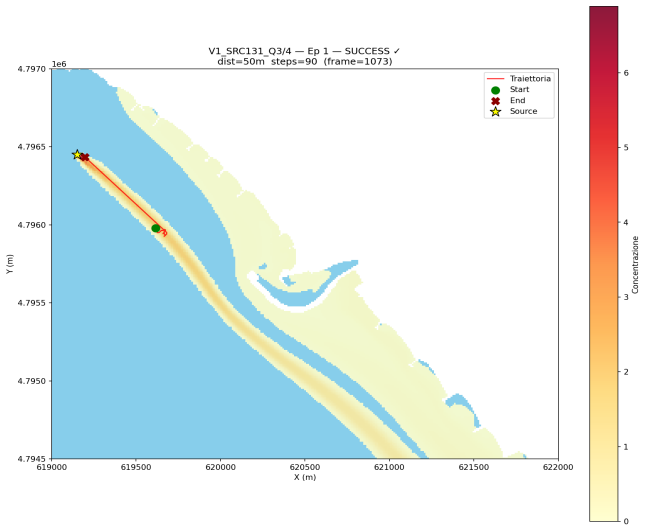
Analisi dei Fallimenti: Durante il training l'agente ha imparato che se procede verso **SE** ha grosse probabilità di trovare la sorgente. Tuttavia, questo porta dei problemi negli scenari V1 e V2 a fine simulazione (Q3/4) perché:
1) **Scenari V1:** Il plume va in direzione opposta tra inizio e fine simulazione (prima verso NW e poi verso SE);
2) **Scenari V2:** Il plume a volte si diffonde a macchia d'olio e diventa troppo disperso per permettere all'agente di trovare la sorgente.

Esempi Caratteristici di Fallimento:

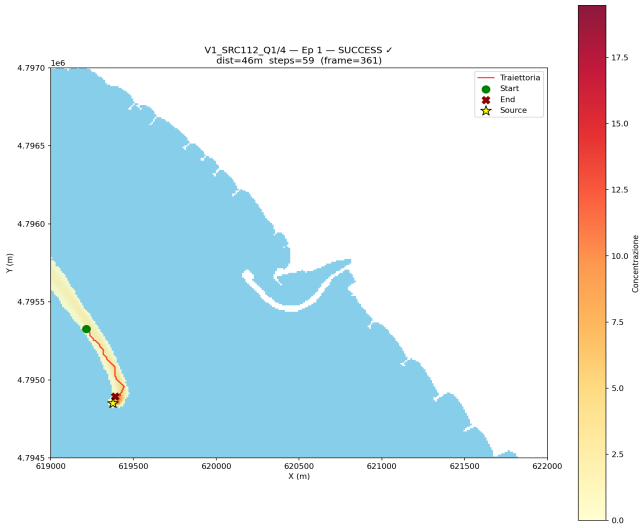
Scenario V1: Cambio di Direzione del Plume



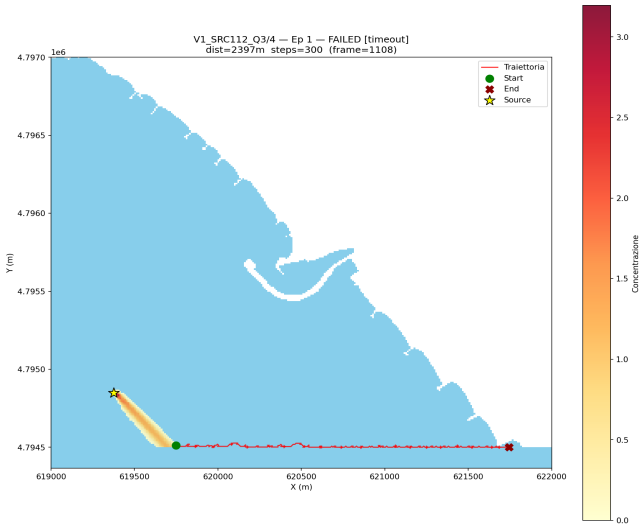
Chunk 0 *(inizio simulazione):*
Plume verso NW

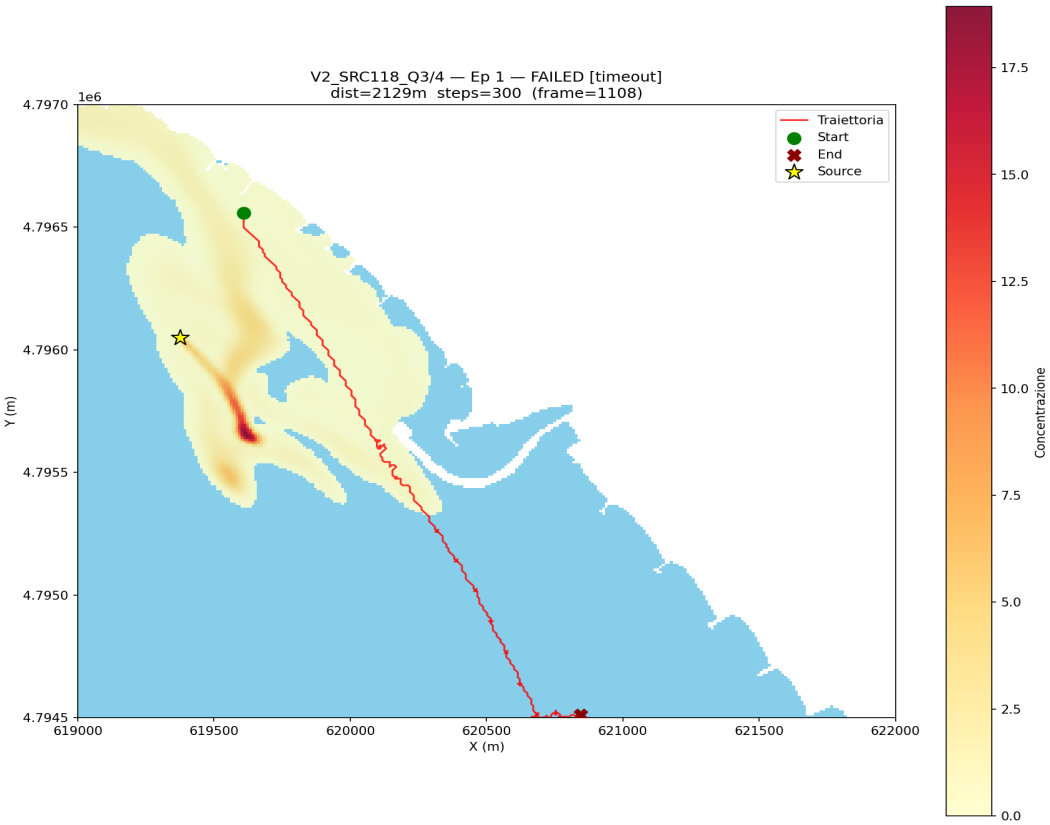


Chunk 2 *(fine simulazione):*
Plume ruota verso SE



Chunk 0 *(inizio simulazione):*
Posizionamento iniziale



Chunk 2 *(fine simulazione):*
Fallimento a trovare la sorgente

Scenario V2: Dispersione Eccessiva del Plume